Ethan Patterson
Nico Cabrera

Algo Hw5 Q1

a.) It starts by creating an adjacency list, determining the neighbors for a given vertex by going through the edge list and listing down the neighbors. From here we start the main function. Keep a curated list of nodes, so we can perform quick index lookups to check if they've been visited or not. Run BFS on the first node in our adj list. The bfs is slightly modified to not include depth as depth does not matter for this problem, just return a list of nodes that are attached. The BFS returns the nodes that can be reached by the input node. From here we can mark each of those nodes in our list of nodes as “visited”. We then run BFS again on the first node not marked as visited, and repeat the same process, doing this until all nodes are “visited”. The answer for “number of connections(roads) to build to make every node accessible from every other node” is the number of BFS runs - 1, as running BFS once, and getting a perfect response with every node being visitable, means we need 0 roads.

b.) Pseudocode:

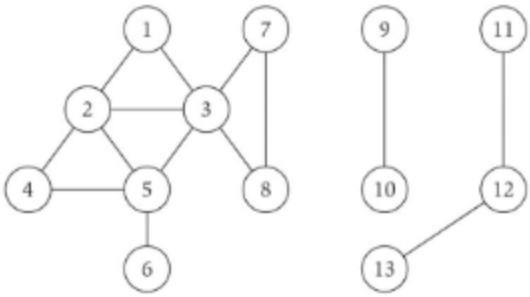
```
Number of edges and vertices is input
List of edges is input
Create adjacency list from list of edges

List of visited vertices is kept, where all values are false

For all vertices:
    If the vertex has not been visited:
        Number_of_bfs_runs +=1
        Create queue
        Enqueue vertex
        Mark vertex as visited
        While queue is not empty:
            Current = queue.pop
            For all neighbors in adjacency_list(current):
                Mark neighbor as visited
                Enqueue neighbor

Output number_of_bfs_runs - 1
```

c.) BFS will always return all the nodes that are reachable from a given node. The number of times we need to run BFS to visit every node will provide us the answer, as long as we rerun BFS on every unvisited node and mark every node that is returned as visited.




Visualize it with the above example. We run BFS on node 1, getting {1, 2, 3, 4, 5, 6, 7, 8} as our output. From our total node list, we know that the only unvisited nodes remaining are { 9, 10, 11, 12, 13}. We run it on the first one here(9) getting { 9, 10} in return. Now we have {11, 12, 13} remaining. Running it on 11 gives us those 3 values, and now we've visited every node. We ran BFS 3 times to create 3 individual graphs, and we'd need 3-1 =2 "roads" to connect them all.

D.) Runtime is O(M+N) where M is num of vertices and N is num of edges.

E.) BFS in and of itself is O(M+N). The adjacency list builder is O(M + N + N) which is O(M + 2N) which is just O(M+N) again. The main edgesToAdd function is just O(M) and the data loading is just O(N), and when adding is still just O(M+N).